

Открываем Claude в интернет.

До этого момента Claude работал только с вашими файлами. Сегодня даём ему доступ к интернету, ставим первые MCP-серверы, подключаем готовые skills из Smithery — и собираем дашборд по реальным ценам акций.

ДЛИТЕЛЬНОСТЬ

6–8 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

WebSearch · WebFetch · MCP · готовые skills · API

ГЛАВНЫЙ ИТОГ

Claude работает с живыми данными, не только с файлами

АРТЕФАКТ

трекер котировок акций + дашборд портфеля по реальным ценам

Здесь больше всего «инсталляций» из всего курса — зато после них Claude получает в руки полный набор инструментов. На следующих этапах ничего нового ставить почти не придётся.

ОБЯЗАТЕЛЬНО • MCP

MCP @playwright/mcp — управление браузером

Что это: Playwright — движок автоматизации браузера от Microsoft. MCP-серверная обёртка даёт Claude инструменты: *открыть страницу, кликнуть, ввести текст, сделать скриншот, прочитать DOM*.

Зачем сейчас: для проверки своего же сайта — Claude открывает ваш дашборд в Chrome и даёт обратную связь. Также для парсинга страниц, на которые нет открытого API.

```
# установка через Smithery
$ npx @smithery/cli@latest install @playwright/mcp --client claude
```

Где взять: smithery.ai/server/@playwright/mcp.

Проверка: в Claude напечатайте `/mcp` — `playwright` в списке. Попросите «*открой через playwright страницу example.com и дай заголовок*» — Claude должен ответить «Example Domain».

ОБЯЗАТЕЛЬНО • MCP

MCP context7 — свежая документация библиотек

Что это: Context7 — сервер от Upstash, через который Claude получает *актуальную* документацию библиотек (chart.js, react, fastapi, всего — тысячи).

Зачем сейчас: модель Claude обучена на данных «до какого-то месяца». За это время библиотеки изменились — чтобы Claude не писал код по устаревшим версиям, мы даём ему живые доки.

```
$ npx @smithery/cli@latest install @upstash/context7-mcp --client claude
```

Где взять: smithery.ai/server/@upstash/context7-mcp.

Проверка: попросите «*через context7 покажи свежий пример chart.js с тёмной темой*» — Claude должен подтянуть документацию и дать рабочий код.

ЖЕЛАТЕЛЬНО • SKILL

Skill excalidraw-diagram-generator — диаграммы и схемы

Что это: skill, который учит Claude генерировать `.excalidraw`-файлы — редактируемые «рукописные» диаграммы. Открываются в excalidraw.com.

Зачем сейчас: когда вы вернётесь к проекту через месяц, проще понять архитектуру по диаграмме, чем по коду. Claude нарисует, как у вас связаны API → CSV → дашборд.

Проверить, нет ли уже: `/skills` в Claude.

```
$ npx @smithery/cli@latest skill add github/excalidraw-diagram-generator --agent claude-code
```

Где взять: smithery.ai/skills/github/excalidraw-diagram-generator.

ОБЯЗАТЕЛЬНО • API-КЛЮЧ

API ключ для котировок: Alpha Vantage

Что это: API-сервис с историческими и текущими ценами акций, валют, крипты.

Бесплатный тариф — до 25 запросов/день.

Зачем сейчас: чтобы дашборд из этапа 02 показывал не «последнюю цену из CSV», а *реальную текущую*.

Где взять: регистрация на alphavantage.co/support/#api-key — одна минута, сразу получаете API-ключ.

Куда положить ключ: создайте файл `.env` в корне проекта (рядом с `CLAUDE.md`):

```
ALPHAVANTAGE_KEY=ваш-ключ-сюда
```

Никогда не коммитьте `.env`: добавьте его в `.gitignore`. Подробно про защиту секретов — этап 05.

Что такое MCP. Почему через него — всё

MCP (Model Context Protocol) — это стандарт «инструментов для AI». Договорились: если кто-то делает сервис, который AI должен уметь использовать, он оформляет его как «MCP-сервер». Любой AI-клиент (включая Claude Code), поддерживающий MCP, мгновенно умеет с этим сервисом работать.

Аналогия: USB. Раньше для каждой мышки нужен был свой драйвер. Сейчас мышка любой марки втыкается в любой компьютер — благодаря стандарту USB. MCP — это USB для AI-инструментов.

Что появляется у Claude после установки MCP-сервера:

- новые «инструменты» (tools), которые он может использовать — от «открой страницу» до «запиши в PostHog»;
- новые «ресурсы» (resources) — ссылки на документы, дашборды, базы знаний;
- иногда — готовые промпты (prompts) для типовых задач сервиса.

В этом этапе мы ставим два MCP: `playwright` (браузер) и `context7` (свежие доки). Этап 09 будет про то, как написать собственный MCP — например, MCP «инвест-помощник» с инструментами `get_price`, `compute_signal`, `set_alert`.

Реестр MCP-серверов и skills — smithery.ai. Это «магазин AI-инструментов»: тысячи серверов от разных авторов, ставятся одной командой.

WebSearch и WebFetch — встроенные «руки в интернет»

Эти два инструмента у Claude Code *уже есть* — ставить ничего не нужно. Разница простая:

- **WebSearch:** «погугли это». Запрос → список ссылок и коротких сниппетов.
- **WebFetch:** «прочитай эту конкретную страницу». URL → текст с неё (HTML очищен).

Типичный сценарий:

```
> найди (WebSearch) последний квартальный отчёт Apple,  
> потом открой (WebFetch) первую ссылку из investor.apple.com  
> и выпиши три ключевых цифры в data/aapl-q.md.
```

Когда уместен WebSearch:

- не знаете точный URL;
- нужно несколько источников;
- задача исследовательская.

Когда WebFetch:

- знаете URL и нужен именно он;
- задача — вытащить данные из конкретной страницы.

WebFetch не подходит для интерактивных страниц (с кучей JavaScript, login и т. д.). Для них — Playwright (см. ниже).

Playwright: Claude управляет настоящим браузером

Playwright — это когда Claude открывает *настоящий* Chrome (через MCP), кликает кнопки, заполняет формы, делает скриншоты. То, чего не умеют WebSearch и WebFetch.

Главные применения в курсе:

1. **Проверка своего сайта.** Сделали дашборд — Claude открывает его, делает скриншот, говорит «здесь график обрезан, вот тут заголовок переносится плохо».
2. **Парсинг сайтов без API.** Например, «сходи на страницу акции на Yahoo Finance, выпиши P/E ratio». Если открытого API нет — парсим браузером.
3. **Тестирование.** «Зарегистрируйся на своём же сайте через тестовый email, проверь что приходит письмо, нажми ссылку из него, проверь что вошёл».

```
> через playwright открой http://localhost:8000/dashboard.html
> в окне 1440x900, сделай скриншот, оцени:
> — нет ли элементов, налезających друг на друга;
> — читаются ли числа в KPI-плитках;
> — все ли графики прорисовались.
```

Context7: чтобы Claude не писал код по старым версиям

Внутри Claude — знания «до даты обучения» (cutoff). За это время многие библиотеки выпустили новые версии, поменяли API. Если Claude помнит «как было», его код может не запуститься.

Context7 решает это так: даёт Claude доступ к *актуальной* документации тысяч библиотек в момент запроса. Claude сам читает свежий API и пишет под него.

Когда вызывать вручную:

- задача завязана на конкретную библиотеку (chart.js, react, fastapi, prisma и т. д.);
- вы видите, что Claude использует устаревший синтаксис;
- вы на «фронтире» технологии — что-то новое.

```
> через context7 подтяни доку chart.js v4
> и сделай тёмную тему графика,
> используя именно текущий API.
```

Когда не нужно: для общих задач (HTML, CSS, базовый JS, общая логика). Там Claude и так точен.

Открытые API: где брать живые финансовые данные

API — это «розетка», в которую программа втыкается, чтобы получить данные. У большинства финансовых сервисов есть открытые API.

Несколько, которые мы используем в курсе:

- **Alpha Vantage** (alphavantage.co) — акции, валюты, крипта. Бесплатно: 25 запросов/день. Нужен API-ключ.
- **Yahoo Finance** (через unofficial API или yfinance) — акции и ETF, без ключа.

- **CoinGecko** (coingecko.com) — цены крипты, без ключа.
- **Frankfurter** (frankfurter.app) — курсы валют от ЕЦБ, без ключа.

Что важно знать про API:

- **Rate limit** — ограничение «сколько запросов в минуту/день». Бесплатные тарифы обычно строгие.
- **API-ключ** — секретный пропуск. *Никогда* не коммитить в git, не писать в HTML, не показывать в скриншотах. Хранить в `.env`.
- **Эндпоинт** — конкретный URL внутри API: например, `/query?function=GLOBAL_QUOTE&symbol=AAPL`.
- **Формат ответа** — обычно JSON.

Готовые skills, которые мы уже ставили

В курсе уже стоят (или встретятся вскоре):

- `frontend-design` — авторские UI без «AI-усреднённости».
- `pdf` — парсинг и заполнение PDF (понадобится в этапе o8b «AI Tax Assistant»).
- `excalidraw-diagram-generator` — диаграммы.
- `playground` — одностраничные интерактивные «исследователи» (понадобится позже).
- `claude-api` — использование Claude как библиотеки внутри ваших скриптов (этап 06 и этап 11).
- `superpowers:brainstorming` и другие из набора `superpowers` — брейнштурм, debugging, TDD-привычки.

Skill vs MCP — в чём разница:

- **Skill** — это набор инструкций и шаблонов, которые Claude *читает и применяет*. Ничего не запускает на вашем компьютере. Просто меняет, как Claude думает и пишет.
- **MCP** — это работающий сервер с инструментами (открыть браузер, прочитать БД, и т. д.). У Claude появляются *новые навыки*.

Когда что использовать — обычно понятно из задачи: «как написать?» — skill; «что-то сделать с внешним миром» — MCP.

PRACTICUM

Прогон № 1 • Smoke test всех новых инструментов

1. Запустите Claude в любой папке. Напечатайте `/mcp` — должны увидеть `playwright` и `context7`.
2. Напечатайте `/skills` — должны увидеть `frontend-design`, `excalidraw-diagram-generator`, остальные.
3. Тест WebSearch: «погугли, какая текущая ставка ФРС, верни одной цифрой и ссылкой».
4. Тест WebFetch: «открой en.wikipedia.org/wiki/Apple_Inc. и дай первый абзац».
5. Тест Playwright: «через `playwright` открой `example.com`, сделай скриншот в окне `1024×768`, опиши, что видно».
6. Тест Context7: «через `context7` подтяни последнюю доку `chart.js`, покажи минимальный пример `bar-chart` с двумя сериями».
7. Тест skill: «через `excalidraw` нарисуй простую диаграмму: `API → CSV → Dashboard`. сохрани в `test.excalidraw`». Откройте файл на `excalidraw.com` — должна быть диаграмма.
8. Если что-то не работает — вернитесь в § 0 и проверьте этот пункт. Не идите дальше с сломанным инструментом.

Что мы тренируем: **уметь проверить установку каждого инструмента отдельно.**

На следующих этапах мы будем полагаться на них постоянно.

Трекер котировок и дашборд по живым ценам

ПРОЕКТ А • ПО ИНСТРУКЦИИ

Трекер котировок акций

Цель: Claude каждый день качает свежие цены пяти акций через Alpha Vantage, складывает в CSV (накопительно), и показывает страничку с графиком за последние 30 дней. Если за сутки цена упала больше 1%, пишет уведомление в `alerts.md`.

Шаги

1. Создайте папку `quotes`, положите туда `.env` с `ALPHAVANTAGE_KEY=...`.
2. Запустите Claude. Сделайте `CLAUDE.md`:

```
> составь CLAUDE.md:
> - стек: python-скрипт fetch.py + одна html-страница
> - данные: data/prices.csv (date, ticker, close)
> - тикеры: AAPL, MSFT, GOOGL, NVDA, VOO
> - источник: alphavantage.co (ключ в .env)
> - .env, .gitignore, data/ – не коммитить
> - стиль страницы – editorial, тёплая палитра
```

3. Просите написать `fetch.py`:

```
> через context7 подтяни актуальный пример
> обращения к Alpha Vantage GLOBAL_QUOTE.
> напиши fetch.py: для каждого тикера из CLAUDE.md
> запрашивает текущую цену, добавляет строку в data/prices.csv.
> ключ – из .env через python-dotenv.
> помни про rate limit 25/день: между запросами sleep 13 секунд,
> чтобы влезть в 5 тикеров.
```

4. Запустите `python fetch.py` один раз. В `data/prices.csv` должны появиться 5 строк за сегодня.

5. Сделайте страницу:

```
> используя skill frontend-design,  
> сделай index.html на основе @data/prices.csv —  
> график за последние 30 дней (chart.js v4 через context7),  
> KPI-плитки сверху: текущая цена, изменение за день, изменение за неделю.  
> стиль — как в финансовой газете (editorial / warm paper).
```

6. Откройте через локальный сервер. Если за один день у вас одна точка на графике — нормально, через неделю появится тренд.

7. Добавьте алерты:

```
> обнови fetch.py: после записи новой цены сравнивай  
> со вчерашней. если изменение меньше -1%, дописывай строку  
> в alerts.md в формате:  
>   - 2026-04-29 NVDA -1.4% (с 110.20 на 108.65)
```

8. Финальный аудит: «перечисли, что сломается, если Alpha Vantage заблокирует мой ключ или у меня кончится rate limit. Что увидит пользователь страницы?»

Что вы здесь освоили

- работу с настоящим API через секретный ключ;
- защиту секретов через `.env` и `.gitignore`;
- использование context7 для актуальной документации;
- маленький скрипт для ежедневного запуска (на этапе 10 поставим на cron).

Дашборд портфеля по живым ценам

Цель: расширение дашборда из этапа 02. Берём `data/operations.csv` из `portfolio-journal` и добавляем загрузку текущих цен через Alpha Vantage. P&L и оценка теперь живые.

Шаги

1. Зайдите в `portfolio-journal`, перенесите туда `.env` с `ALPHAVANTAGE_KEY`. Обновите `.gitignore`.
2. Обновите `CLAUDE.md`: добавьте раздел про `ALPHAVANTAGE_KEY` и что цены теперь загружаются с API.
3. Просите Claude доработать `dashboard.html`:

```
> в dashboard.html замени «последняя цена из CSV»  
> на загрузку текущей цены через Alpha Vantage.  
> для каждого уникального тикера из @data/operations.csv –  
> один запрос. результаты кэшируй в data/prices-cache.json  
> со временем жизни 15 минут (чтобы не упереться в rate limit).  
> в KPI-плитках добавь маркер «обновлено N минут назад».
```

4. Откройте дашборд. P&L теперь живой — меняется при каждой перезагрузке.
5. Уточняйте:
 - «подсвечивай нереализованный P&L зелёным/красным — в зависимости от знака»;
 - «добавь блок «алерты» — читать из `../quotes/alerts.md`, показывать последние 5»;
 - «если ключ Alpha Vantage не задан, покажи мягкое предупреждение и используй последнюю цену из CSV».
6. Используйте **Playwright** для ревизии:

```
> через playwright открой http://localhost:8000/dashboard.html  
> в окне 1440x900. сделай скриншот, расскажи:  
> – все ли цены загрузились,  
> – нет ли элементов налезających друг на друга,  
> – читаются ли числа в KPI.
```

7. Финальный аудит: «какие три риска у этого подхода (живые цены через сторонний API)? как от них защититься?»

Что вы здесь освоили

- совмещение локальных данных (CSV) с живыми (API);
- кэширование, чтобы не упереться в rate limit;
- использование Playwright для собственного контроля качества;
- graceful fallback — что показать, если внешний сервис недоступен.

Те же приёмы, что в А и В — новые источники данных. CoinGecko и RSS не требуют API-ключа, поэтому проще, чем Alpha Vantage.

ПРОЕКТ С • САМОСТОЯТЕЛЬНО

Трекер криптовалют

Цель: вариация проекта А — те же шаги (*fetch* + страница + *alerts*), но вместо акций криптовалюты через CoinGecko (бесплатно, без ключа).

Подсказки

- Источник: `api.coingecko.com`, эндпоинт `/api/v3/simple/price`.
- Список: BTC, ETH, SOL, BNB, ADA (или ваш).
- Без API-ключа, но rate limit — около 30/мин.
- Алерты делайте на колебание **больше 5%** за сутки (крипта волатильнее).
- Дизайн — в противоположной стилистике от трекера акций (например, dark + неон, если акции были editorial).

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ есть отдельная папка `crypto` со своим `CLAUDE.md` (без API-ключа);
- ✓ `fetch` скрипт работает запуском `python fetch.py`, добавляет данные в `data/prices.csv`;
- ✓ страница показывает 5 криптовалют, графики 30 дней, KPI;
- ✓ порог алерта — 5%, не 1% (крипта волатильнее);
- ✓ дизайн отличается от стиля трекера акций;
- ✓ через playwright Claude сделал ревизию страницы.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Сравни мой трекер крипты с трекером акций. Где код одинаковый и можно вынести в общую функцию? Если бы ты писал «универсальный трекер цен» — на какие три источника он должен уметь работать сразу?»

Дайджест финансовых новостей через RSS

Цель: вариация проекта А — вместо API цен берём RSS-ленты финансовых сайтов. Получаем сводный утренний дайджест.

Подсказки

- Источники: 3–4 ленты на ваш вкус. Например, FT Markets, Reuters Markets, Bloomberg Markets.
- Структура та же: `fetch.py` + страница + накопление в CSV/JSON.
- Claude должен *саммаризировать* заголовки в три-пять ключевых тем дня (используйте обычный диалог Claude, без API).
- На странице — «утренний дайджест» (топ-темы) и ниже сырые заголовки со ссылками на оригиналы.
- Алерт-секция: если в заголовках упоминаются ваши тикеры (из `portfolio-journal/data/operations.csv`) — выделить их.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ есть папка `news-digest` с `CLAUDE.md` и списком фидов;
- ✓ `fetch` скрипт скачивает заголовки из 3+ лент и складывает в `data/news.csv`;
- ✓ страница показывает 3–5 «тем дня» (саммари) и ниже сырые заголовки со ссылками;
- ✓ работает выделение упоминаний ваших тикеров из `portfolio-journal`;
- ✓ через playwright проведена ревизия;
- ✓ в `CLAUDE.md` зафиксировано: где живут фиды и как добавить новый.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Сделай мини-аудит UX через playwright: открой страницу, представь, что у меня 30 секунд утром у кофе. За 30 секунд чтения я узнаю что-то полезное про мой портфель? Что мешает? Что вынес бы вверх?»

Что добавилось к словарю

MCP	Model Context Protocol — стандарт «инструментов для AI». Через MCP-сервер Claude получает новые навыки: открывать браузер, читать свежие доки, общаться с внешним сервисом. Аналогия — USB для AI.
Smithery	Реестр MCP-серверов и skills. Тысячи готовых пакетов, ставятся одной командой <code>npm install @smithery/cli@latest</code> <code>install/skill add ...</code> . Главное место «магазина AI-инструментов».
WebSearch	Встроенный инструмент Claude Code: «погугли это». Возвращает список ссылок и коротких сниппетов. Не требует установки.
WebFetch	Встроенный инструмент: «прочитай эту страницу». На вход URL, на выход — очищенный текст. Не подходит для интерактивных страниц — для них Playwright.
Playwright	Движок автоматизации браузера от Microsoft. Через MCP даёт Claude инструменты: открыть страницу, кликнуть, ввести текст, сделать скриншот. Используется для ревизии своих сайтов и парсинга страниц без API.
Context7	MCP-сервер от Upstash, дающий Claude доступ к <i>актуальной</i> документации тысяч библиотек. Решает проблему «обучен на старых версиях».
API	Application Programming Interface — «программный интерфейс»: способ одной программы получить данные у другой по строгим правилам. Финансовые сервисы (Yahoo, Alpha Vantage, CoinGecko) предоставляют открытые API для цен.
API-ключ	Секретная строка, которая идентифицирует вас перед API. Никогда не коммитьте в git, не пишите в HTML, не показывайте в скриншотах. Хранится в <code>.env</code> , который добавляется в <code>.gitignore</code> .
RSS	Really Simple Syndication — стандарт «лент новостей» в формате XML. У большинства новостных сайтов есть открытые RSS-фида — самый простой способ собирать заголовки автоматически.
Rate limit	Ограничение, сколько запросов в единицу времени можно сделать к API. Бесплатные тарифы обычно строгие. Решения: кэширование, паузы между запросами, графический fallback.
Эндпоинт	Конкретный URL внутри API, отвечающий за одну функцию. Например, у Alpha Vantage есть эндпоинт <code>/query?function=GLOBAL_QUOTE&symbol=AAPL</code> для текущей цены акции.
JSON	JavaScript Object Notation — формат обмена данными между программами.

Похож на «вложенные словари». Большинство API возвращают ответ именно в JSON.

.env

Текстовый файл с секретными переменными окружения (API-ключи, пароли). Лежит в корне проекта, обязательно в .gitignore. Программы читают его через библиотеки типа python-dotenv.

Семь вопросов на закрепление

01 В чём разница между skill и MCP?

Skill — набор инструкций и шаблонов, который Claude читает и применяет. Меняет, как он думает и пишет. Ничего не запускает на вашем компьютере. **MCP** — работающий сервер с инструментами (открыть браузер, прочитать БД). Даёт Claude *новые навыки*.

02 Когда лучше WebFetch, а когда Playwright?

WebFetch — для статичных страниц с текстом (Wikipedia, документация, блоги). Быстрее и легче. **Playwright** — для интерактивных (с тяжёлым JavaScript, авторизацией, динамической подгрузкой). Также для ревизии своих собственных сайтов — со скриншотами и кликами.

03 Зачем нужен Context7, если Claude и так знает большинство библиотек?

Знания Claude обрываются на дате обучения. За это время многие библиотеки выпускают новые версии и меняют API. Context7 даёт Claude *актуальную* документацию в момент запроса — и он пишет под текущий API, а не под тот, что помнит.

04 Где хранить API-ключ Alpha Vantage и что нельзя делать с этим файлом?

В файле `.env` в корне проекта (например, `ALPHAVANTAGE_KEY=xxx`). **Никогда** не коммитить в git, не вшивать в HTML/JS, не показывать в скриншотах. Файл должен быть в `.gitignore`.

05 Что такое rate limit и три способа в него не упереться?

Rate limit — ограничение, сколько запросов в минуту/день можно сделать к API. Способы: **кэширование** (запоминать ответы на N минут), **паузы между запросами** (sleep), **graceful fallback** — если ответа нет, показать что-то полезное (например, последнюю кэшированную цену с пометкой «устаревшая»).

06 Какой командой проверить, какие MCP сейчас подключены к Claude Code?

`/mcp` внутри Claude Code (slash-команда). Аналогично `/skills` — для skills.

07 Зачем использовать Playwright для ревизии *своего* же сайта?

Чтобы Claude посмотрел на сайт глазами пользователя — не по коду, а по скриншоту. Видит ли он все элементы, читаются ли числа, не наезают ли блоки друг на друга. Это вторая, независимая «пара глаз» — катастрофически полезно, потому что код может быть «правильным», но UX — плохим.

Что я теперь умею

- ☐ Поставил MCP `@playwright/mcp` и `@upstash/context7-mcp`, проверил через `/mcp`.
- ☐ Подтвердил наличие skills: `frontend-design`, `excalidraw-diagram-generator` через `/skills`.
- ☐ Знаю разницу WebSearch / WebFetch / Playwright и когда что уместно.
- ☐ Получил Alpha Vantage API key, сохранил в `.env`, добавил `.env` в `.gitignore`.
- ☐ Понимаю, что такое rate limit, и знаю три способа в него не упереться.
- ☐ Различаю skill и MCP: skill меняет «как Claude думает», MCP даёт ему новые навыки.
- ☐ Сделал четыре проекта: трекер котировок, обновлённый дашборд по живым ценам, трекер крипты, дайджест новостей.

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«Один MCP — и Claude умеет открыть браузер. Один skill — и он пишет в вашей стилистике. Сложность — в выборе, не в установке».

Этап 03 пройден — у Claude теперь есть руки в интернет. Дальше учим его думать перед действием на крупных задачах.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 02
- Этап 04